CVBW-DeepFit — Reference Implementation & Implementation Process

Companion code for the manuscript *CVBW-DeepFit: Audiogram-Conditioned Deep Denoising Fused with a Differentiable Continuously Variable Bandwidth Filter Bank for Sub-5 ms, Hardware-Efficient Hearing Aid Processing.*

This is a **research reference implementation**: it is written for clarity and to prove each novelty claim is realizable, not for production throughput. Every module has a runnable self-test.

---

1. What each novelty looks like in code

Manuscript contribution	Where it lives	How to see it
#1 Filter-bank fusion (one analysis/synthesis)	`model.py` — mask and fitting gain merged in one sub-band domain, single `CVBWSynthesis` pass	`python -m cvbw_deepfit.model`
#2 Differentiable Farrow bandwidth factor	`cvbw_filterbank.py` — `rf` is an `nn.Parameter`; gradient flows through Lagrange sub-filters	filterbank self-test prints `grad on rf`
#3 Audiogram-conditioned suppression (FiLM)	`mask_estimator.py` — `AudiogramEncoder` + `FiLM` in every residual block	mask changes when audiogram changes
#4 HI-pMOS perceptual objective	`losses.py` — `hipmos_net` hook (stand-in provided)	swap in trained estimator
#5 Multi-objective SW/HW optimization	`losses.py` (soft HW penalty) + `quantize.py` (MOABC + SID-CSE)	`python -m cvbw_deepfit.quantize`
#6 Common-currency HW benchmark	`quantize.py` — adder count; export RTL from coefficients	Pareto front of attn vs adders

---

2. Quick start

```bash
```

```
pip install -r requirements.txt          # torch, numpy, scipy

# prove each component works
python -m cvbw_deepfit.cvbw_filterbank    # differentiable bank + rf gradient
python -m cvbw_deepfit.mask_estimator     # FiLM conditioning + param count
python -m cvbw_deepfit.model              # full fused forward/backward
python -m cvbw_deepfit.losses             # multi-objective loss
python -m cvbw_deepfit.quantize           # MOABC Pareto front (attn vs adders)

# end-to-end training smoke test on synthetic data
python -m cvbw_deepfit.train --smoke
```

Expected: the fused model reports ~130k parameters (well under the 250k budget), and the smoke test prints decreasing loss components.

---

3. Implementation process (the recommended build order)

The manuscript's nine phases map onto a concrete engineering sequence. Build and validate each stage before moving on — the whole point of the fused design is that a bug in the filter bank corrupts both denoising and fitting.

Stage 0 — Environment & data pipeline

1. Mixed-corpus loader yielding `(noisy, clean, audiogram, target_fit_db)`. Speech: VoxCeleb2 / LibriVox / VCTK. Noise: WHAM!/WHAMR!, DEMAND, AudioSet.
2. SNR sampler: `N(8, 7)` dB (realistic) and `N(0, 7)` dB (hard). RMS-normalise to −20 dBFS before mixing. Enforce speaker/noise/language train-test disjointness.
3. Audiogram sampler drawing from a clinical distribution so the conditioned model interpolates across the ability spectrum (this is what removes the NH penalty — the model must *see* NH-like audiograms during training).

Stage 1 — Fixed prototype filter (`prototype_lowpass`)

Replace the windowed-sinc placeholder with a `scipy.signal.remez` design at the paper specs (passband ripple 0.05 dB, stopband 80 dB, transition 0.1625). Validate the magnitude response before anything else touches it.

Stage 2 — Differentiable CVBW bank (`cvbw_filterbank.py`)

1. Build the order-4 Lagrange Farrow sub-filters (`farrow_subfilters`).
2. Implement the SRC interpolator and confirm `rf` gradient is non-zero.
3. Unit-test bandwidth control: feed a unit impulse, confirm the main-lobe width scales with `rf` (the CVBW property from the reference filter paper).

Stage 3 — Mask estimator (`mask_estimator.py`)

1. Causal TCN; verify no future-sample leakage (shift-invariance test).
2. Wire FiLM; **assert the audiogram actually changes the output** (self-test does).
3. Keep params ⩽ 250k as a hard constraint for the later NAS.

Stage 4 — Fusion & synthesis (`model.py`)

Merge complex mask with sub-bands, apply per-band fitting gains from the audiogram head, run one synthesis pass, blend 90/10 for environmental awareness.

Stage 5 — Multi-objective loss (`losses.py`)

Start with `L_cIRM` only; add terms one at a time and watch each metric. Replace the SI-SDR/envelope stand-ins with the trained HI-pMOS network and a HASPI surrogate (Clarity tools) once Stage 6 is done.

Stage 6 — HI-pMOS estimator (separate training job)

1. Collect stratified ratings (NH crowdworkers + in-clinic HI/CI).
2. MLP maps objective features → MOS; label ~600k unrated files.
3. Train the intrusive estimator (STFT+MEL, clean+degraded); fine-tune on real HI/CI ratings. Validate correlation vs measured SRT benefit against STOI/PESQ.

Stage 7 — NAS + quantization (`quantize.py`)

1. Evolutionary NAS over layer type/kernel/filters/dilation/depth; fitness = Δ(HI-pMOS) under latency + param constraints.
2. CSD-quantize coefficients; run MOABC (colony 100, archive 400, cycles 250, limit 5) to recover stop-band attenuation.
3. Modified SID-CSE (shift range [−18, +18], directed minimal spanning tree) to minimise adders. The provided compact MOABC reproduces the attn-vs-adder Pareto trade-off at reduced scale.

Stage 8 — Hardware export

1. Emit Verilog from the multiplier-less coefficient tree.
2. Synthesise on Xilinx Kintex-7 (XC7K70T) for comparability, plus a low-power target. Report multipliers/band, adders, slice LUTs, registers, mW, max clock.
3. **Latency-accounting table**: separate algorithmic look-ahead, filter group delay, buffering, AD/DA. This table is itself a contribution — the literature is inconsistent about what "1 ms" or "23 ms" includes.

Stage 9 — Evaluation

Objective (Comparison Table 1), fitting quality (Table 2), behavioral SRT + pupillometry (Table 3), ablations (Table 4). Use TOST equivalence testing for the "HI-with-processing ≈ NH-without-processing" claim rather than a non-significant p-value.

4. Files

```
cvbw_deepfit/
├── cvbw_filterbank.py   # differentiable Farrow CVBW analysis/synthesis
├── mask_estimator.py    # FiLM-conditioned causal TCN complex-mask estimator
├── model.py             # fused CVBW-DeepFit model
├── losses.py            # multi-objective loss (cIRM, HI-pMOS, HASPI, match, hw,
TFS)
├── train.py             # training loop + synthetic smoke test
├── quantize.py          # CSD + MOABC + adder-count (SID-CSE hook)
└── __init__.py
requirements.txt
README.md
```

5. Honest limitations of this reference code

- The prototype filter is a windowed sinc, not a remez design — swap before reporting filter metrics.
- HI-pMOS and HASPI are differentiable stand-ins; real experiments need the trained estimator and Clarity's HASPI.
- The SRC interpolator prioritises readability; a production version would use the polyphase realisation and fixed-point arithmetic.
- MOABC here applies word-length/SPT globally; the paper's version optimises per-coefficient with the full directed-minimal-spanning-tree SID-CSE.
- No target-speaker selection — like the systems it builds on, the model treats all prominent talkers as "speech."